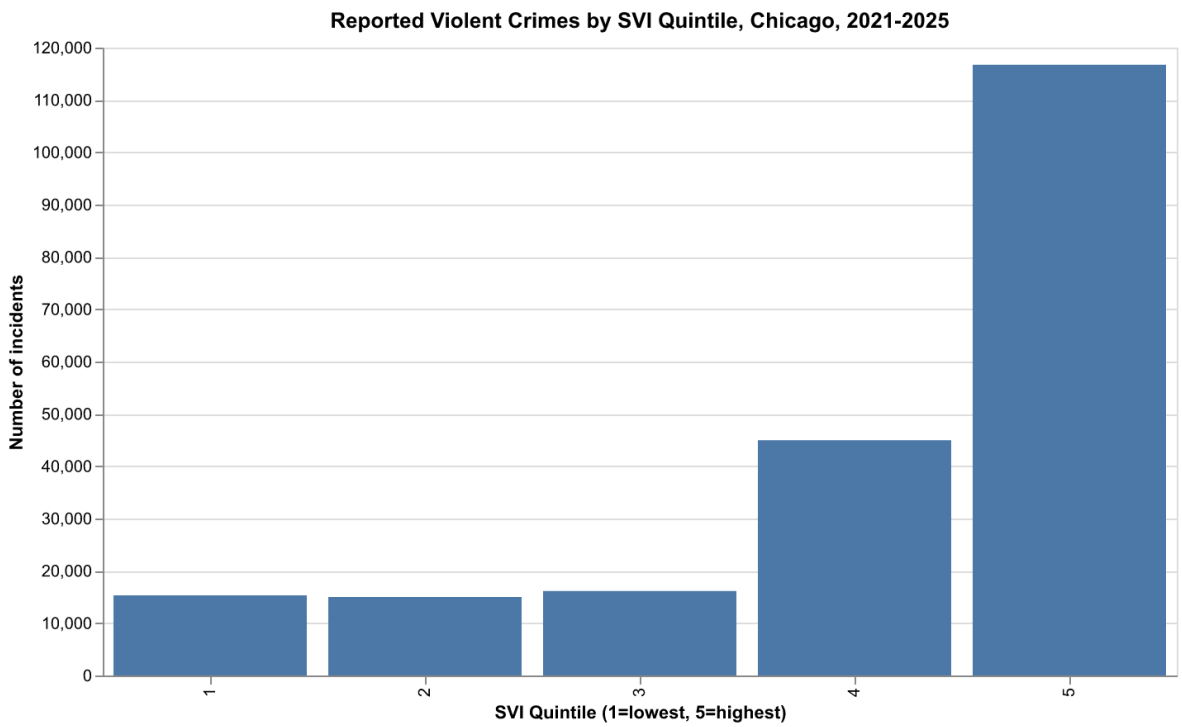
SVI Plots

```
#libraries and plugins
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import altair as alt
import os
import seaborn as sns
import geopandas as gpd
from shapely.geometry import Point
import json

#settings
pd.set_option('display.max_columns', None)
pd.set_option("display.max_colwidth", None)
pd.set_option("display.width", None)

#extra settings
import warnings
warnings.filterwarnings('ignore')
alt.renderers.enable("png")
alt.data_transformers.disable_max_rows()

import tempfile
from IPython.display import SVG, display, Image
import vl_convert as vlc

def display_altair_png(chart, scale=2):
    """
    Render an Altair chart to a PNG and display it inline.

    Parameters
    ----------
```

```
    chart : altair.Chart
        Altair chart object to render.
    scale : int, optional
        Resolution scaling factor for the PNG (default = 2). Use scale=2 for standard slides
    """
    png_bytes = vlc.vegalite_to_png(chart.to_dict(), scale=scale)

    # Write to a temporary PNG file and display
    with tempfile.NamedTemporaryFile(suffix=".png") as tmp:
        tmp.write(png_bytes)
        tmp.flush()
        display(Image(filename=tmp.name))
```

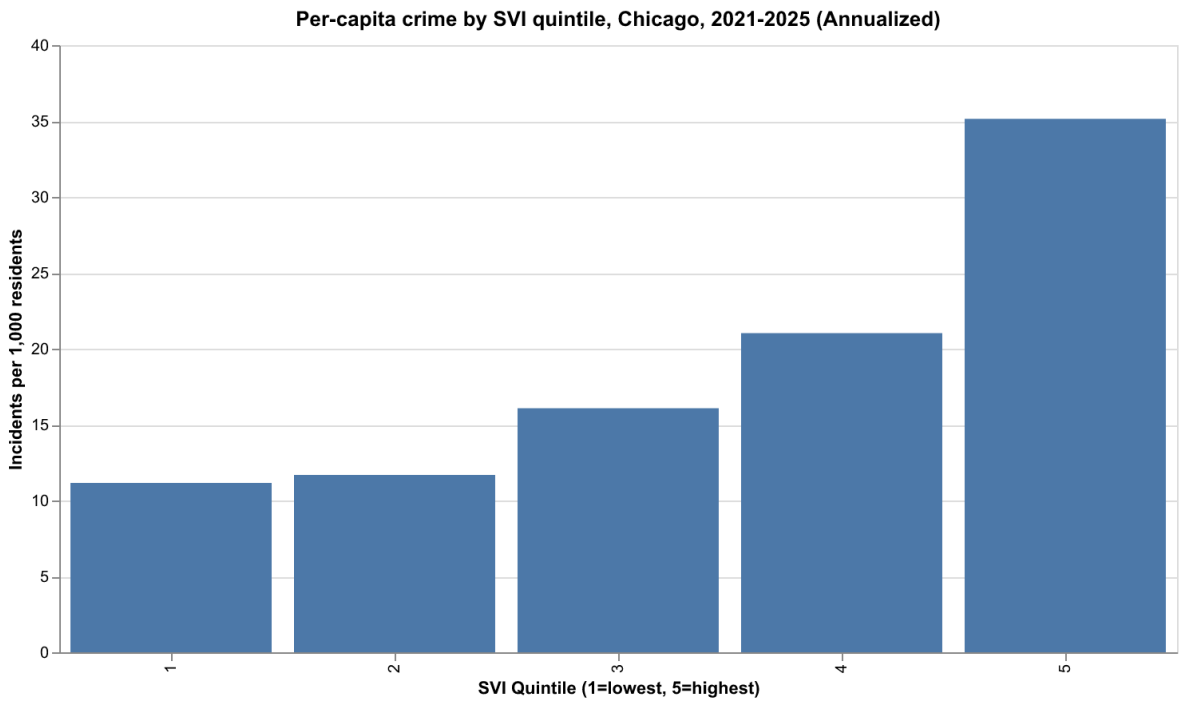
```
#loading and adjusting data
base_path = os.getcwd()
crime = pd.read_csv(os.path.join(base_path, "data/derived-data/crime_svi_merge_no_na.csv"))
tracts = gpd.read_file(os.path.join(base_path, "data/raw-data/Census_Tracts_20260228/geo_expo
crimes_with_tracts = gpd.read_file(os.path.join(base_path, "data/derived-data/crimes_with_tra
tract_full = gpd.read_file(os.path.join(base_path, "data/derived-data/tract_full.geojson"))

#adding and fixing date(s)
crime["Date"] = pd.to_datetime(crime["Date"], errors="coerce")
crime["month"] = crime["Date"].dt.to_period("M").dt.to_timestamp()

#adding quintiles for SVI and in tract_full
crime["SVI_quintile"] = pd.cut(
    crime["SVI"],
    bins=[0, 0.2, 0.4, 0.6, 0.8, 1.0],
    labels=["1", "2", "3", "4", "5"],
    include_lowest=True)

#tract full
tract_full["SVI_quintile"] = pd.cut(
    tract_full["svi"],
    bins=[0, 0.2, 0.4, 0.6, 0.8, 1.0],
    labels=["1", "2", "3", "4", "5"],
    include_lowest=False)
#replace NA with 0 that are ignored
tract_full["pop"] = tract_full["pop"].fillna(0)
```

```
#reported crimes by quintile, overall
crime = crime.dropna(subset=["SVI_quintile"])
incidents_q = alt.Chart(crime).mark_bar().encode(
    x=alt.X("SVI_quintile:N", title="SVI Quintile (1=lowest, 5=highest)", sort=["1","2","3",
    y=alt.Y("count():Q", title="Number of incidents")
).properties(
    title="Reported Violent Crimes by SVI Quintile, Chicago, 2021-2025",
    width=650,
    height=380
)
display_altair_png(incidents_q)
```



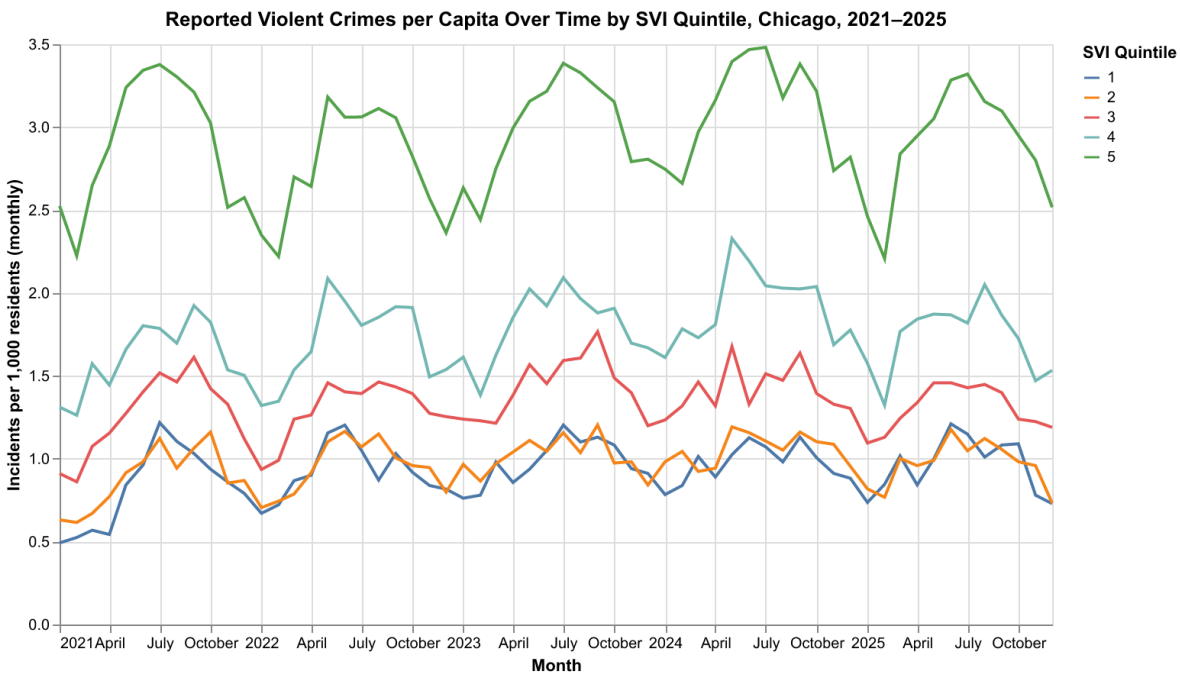

```
#report crimes by quintile, per capita annualized
pop_by_q_full = (
    tract_full.dropna(subset=["pop", "SVI_quintile"])
              .groupby("SVI_quintile", as_index=False, observed=True)
              .agg(pop=("pop", "sum")))
crime_q = (
    crime.dropna(subset=["SVI_quintile"])
         .groupby("SVI_quintile", as_index=False, observed=True)
         .agg(incidents=("ID", "count")))
```

```
q_rate_full = crime_q.merge(pop_by_q_full, on="SVI_quintile", how="right")
q_rate_full["incidents"] = q_rate_full["incidents"].fillna(0)
q_rate_full["incidents_per_1000"] = ((q_rate_full["incidents"] / 5)/ q_rate_full["pop"]) * 1
crime_q_plot_full = alt.Chart(q_rate_full).mark_bar().encode(
    x=alt.X("SVI_quintile:N", title="SVI Quintile (1=lowest, 5=highest)", sort=["1","2","3",
    y=alt.Y("incidents_per_1000:Q", title="Incidents per 1,000 residents")
).properties(
    title="Per-capita crime by SVI quintile, Chicago, 2021-2025 (Annualized)",
    width=700,
    height=380
)
display_altair_png(crime_q_plot_full)
```



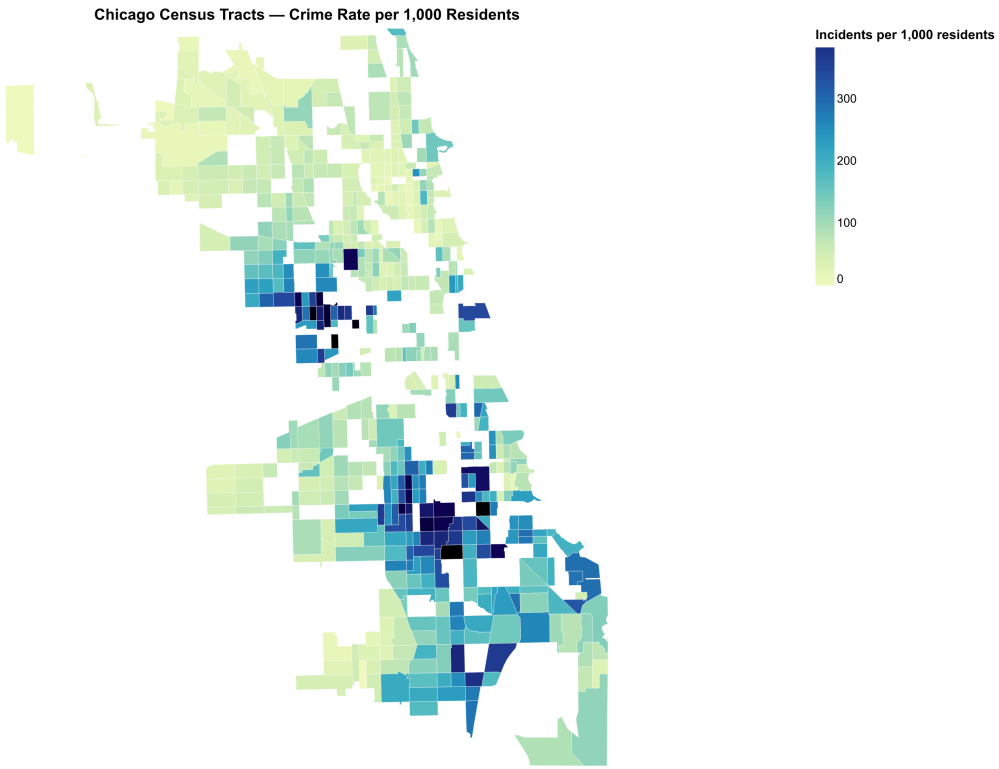

```
#violent crime rate over time, by quintile, per capita
incidents_per_month_q = (
    crime.dropna(subset=["month", "SVI_quintile"])
        .groupby(["month", "SVI_quintile"], as_index=False, observed=True)
        .agg(incidents=("ID", "count")))
incidents_per_month_q = incidents_per_month_q.merge(pop_by_q_full, on="SVI_quintile", how="le
incidents_per_month_q["incidents_per_1000"] = (incidents_per_month_q["incidents"] / incidents
incidents_over_time = alt.Chart(incidents_per_month_q).mark_line().encode(
```

```
    x=alt.X("month:T", title="Month"),
    y=alt.Y("incidents_per_1000:Q", title="Incidents per 1,000 residents (monthly)"),
    color=alt.Color("SVI_quintile:N", title="SVI Quintile", sort=["1","2","3","4","5"])
).properties(
    title="Reported Violent Crimes per Capita Over Time by SVI Quintile, Chicago, 2021-2025"
    width=650,
    height=380)
display_altair_png(incidents_over_time)
```



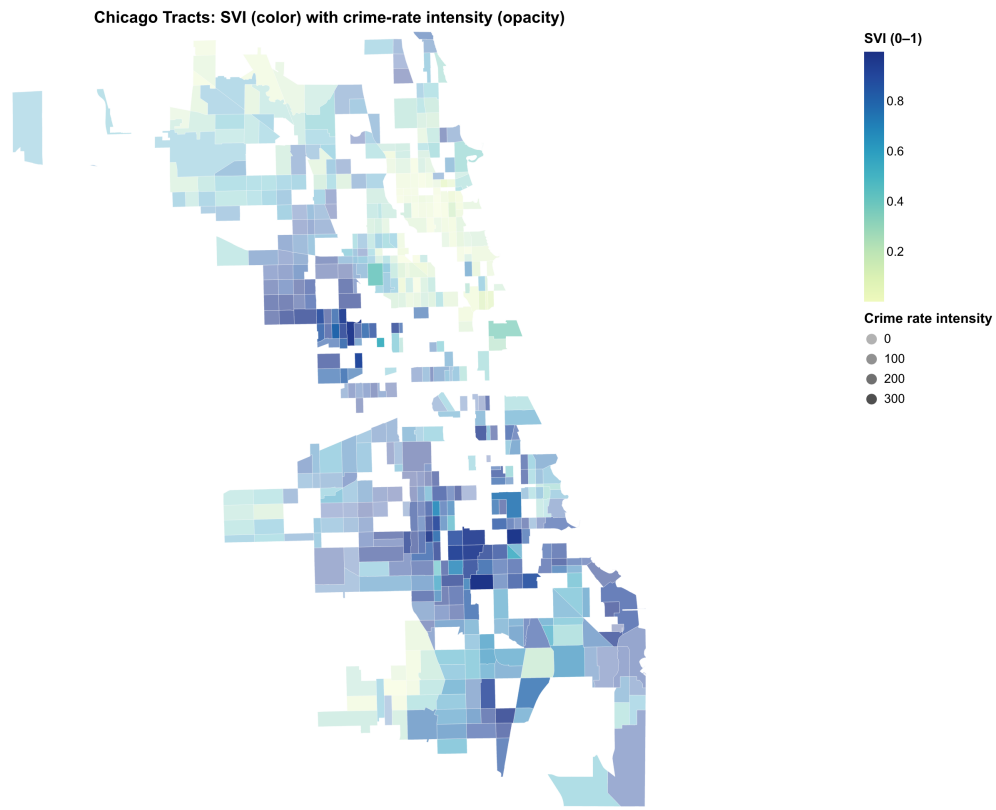

```
cap = float(tract_full["crime_per_1000"].quantile(0.95))
tract_geo = json.loads(tract_full.to_json())
crime_rate_map = (
    alt.Chart(alt.Data(values=tract_geo["features"]))
    .mark_geoshape(stroke="white", strokeWidth=0.25)
    .encode(
        color=alt.Color(
            "properties.crime_per_1000:Q",
            title="Incidents per 1,000 residents",
            scale=alt.Scale(domain=[0, cap])
        )
    )
    .properties(width=820, height=620, title="Chicago Census Tracts - Crime Rate per 1,000 Re
```

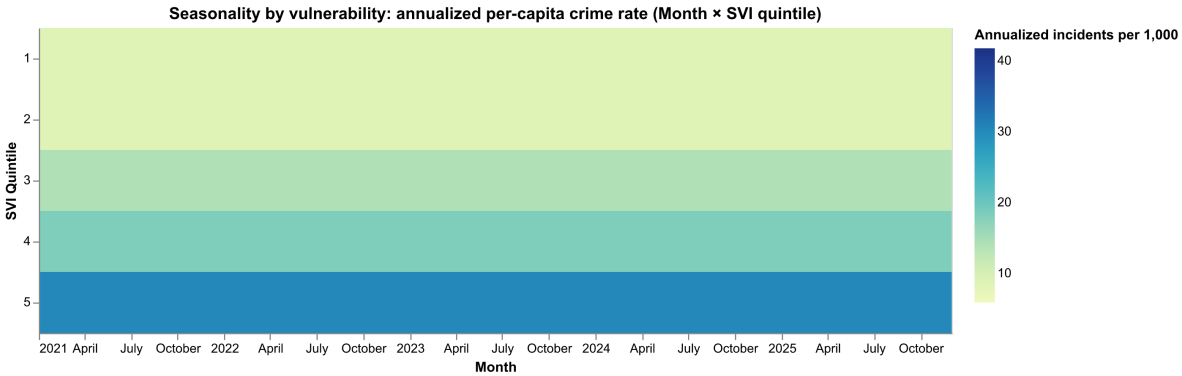
```
    .project(type="mercator")
)
display_altair_png(crime_rate_map, scale=3)
```




```
cap = float(tract_full["crime_per_1000"].quantile(0.95))

tract_geo = json.loads(tract_full.to_json())

map_svi_crime = (
    alt.Chart(alt.Data(values=tract_geo["features"]))
    .mark_geoshape(stroke="white", strokeWidth=0.2)
    .encode(
        color=alt.Color("properties.svi:Q", title="SVI (0-1)"),
        opacity=alt.Opacity(
            "properties.crime_per_1000:Q",
            title="Crime rate intensity",
            scale=alt.Scale(domain=[0, cap])))
    .properties(
        width=820,
```

```
        height=620,
        title="Chicago Tracts: SVI (color) with crime-rate intensity (opacity)"
    )
    .project(type="mercator")
)
display_altair_png(map_svi_crime, scale=3)
```




```
month_q = (
    crime.dropna(subset=["month", "SVI_quintile"])
        .groupby(["month", "SVI_quintile"], as_index=False, observed=True)
        .agg(incidents=("ID", "count")))
month_q = month_q.merge(pop_by_q_full, on="SVI_quintile", how="left")
month_q["incidents_per_1000_month"] = (month_q["incidents"] / month_q["pop"]) * 1000
month_q["incidents_per_1000_annualized"] = month_q["incidents_per_1000_month"] * 12
heatmap = alt.Chart(month_q).mark_rect().encode(
    x=alt.X("month:T", title="Month"),
    y=alt.Y("SVI_quintile:N", title="SVI Quintile", sort=["1","2","3","4","5"]),
    color=alt.Color("incidents_per_1000_annualized:Q", title="Annualized incidents per 1,000
```

```
).properties(
    title="Seasonality by vulnerability: annualized per-capita crime rate (Month × SVI quinti
    width=720,
    height=240
)
display_altair_png(heatmap, scale=3)
```